1310EE370000 Machine Learning

Final project

第六組

組長：蔡昀諺

組員：蕭丞凱 王彥凱

1. Introduction

自動光學檢測（Automated Optical Inspection，簡稱 AOI）是一種高速度與高精度的光學影像檢測系統，其系統的開發是源於人工檢查就算有光學儀器的幫助還是可能有效率低下，精度不穩定，人為錯誤（包括受到工人精神狀態、漏檢、誤檢或錯誤判斷等影響）。AOI 技術的應用範圍涵蓋高科技產業的研發、製造質量控制、國防、民生、醫療、環保和電力等各個領域。本次期末專案採用基於 PyTorch 框架的 VGG 模型並成功進行了圖像處理和分類。我們期望展示 VGG 模型對於自動光學檢測技術的強大潛力，在提升產品質量的同時，也能夠減少人工檢查的時間和錯誤率。

2. Method

在這次的 project 中，我們使用了 deep learning 技術中的 VGG 模型的簡易版進行圖像辨識，我們先定義了數據集並將訓練圖像進行加載並進行預處理（包括轉為灰階與調整圖像大小）。在定義好 VGG 的架構後使用 PyTorch 來訓練模型，訓練過程會分為 100 個 epoch，我們將監控訓練過程中的 loss function 來評估模型的性能。在模型訓練結束後保存模型參數用於後續的檢測。

VGG：

VGG 是一種特定設計的 CNN 架構，具有更深的層數和標準化的設計。以 VGG16 為例，有 13 層卷積層（Convolution Layer），皆是 3X3 的卷積核；3 層全連結層，分別為 4096、4096、1000 的節點數；5 層最大池化層，2X2 的最大池化層，stride 設定為 2。有權重的隱藏層總計有 16 層，對應模型名稱中的 16。

VGG 較深的模型架構，可以加強深度學習透過激勵函數（Activation Function）所帶來的非線性結構，這樣的結構也是深度學習相較於傳統統計模型的一大優勢，能擷取出更多有用的訊息。

由模型架構可以發現，VGG 模型皆使用 3X3 的卷積核，因為相較於較大的卷積核，更換成 3X3 的卷積核可以使模型架構更深，並且參數量還更少。

3. Code Explanation

```
import os
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim

from PIL import Image
from tqdm import tqdm
from torchvision import transforms
from torch.utils.data import Dataset, DataLoader
```

os: 用於處理路徑。
pandas: 用於讀取 CSV 文件並處理表格數據。
torch:用於構建神經網絡。
torch.nn: 包含神經網絡構建模組。
torch.optim: 優化器。
PIL.Image: 用於讀取圖像。
tqdm: 顯示進度條。
torchvision.transforms: 用於圖像預處理。
torch.utils.data: 提供數據集和數據加載器相關模組。

```
class GrayscaleDataset(Dataset):
    def __init__(self, csv_file, root_dir, transform=None):
        self.data_frame = pd.read_csv(csv_file)
        self.root_dir = root_dir
        self.transform = transform

    def __len__(self):
        return len(self.data_frame)

    def __getitem__(self, idx):
        img_name = os.path.join(self.root_dir, self.data_frame.iloc[idx, 0])
        image = Image.open(img_name).convert('L')  # Convert to grayscale
        label = int(self.data_frame.iloc[idx, 1])

        if self.transform:
            image = self.transform(image)
        return image, label
```

__init__:
- 初始化資料集。
- 從 csv_file 讀取圖像路徑及標籤，儲存在 self.data_frame。
- root_dir 是圖像存儲的根目錄。
- transform 是圖像的預處理方式。

__len__:
- 返回資料集中樣本的數量。

__getitem__:
- 根據索引 idx，從 CSV 檔案中獲取圖像名稱及其標籤。

- 拼接圖像路徑並讀取圖像，轉換為灰度模式。
- 如果指定了 transform，則對圖像進行處理。
- 返回圖像及其對應的標籤。

```
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor()
])
```

定義圖像預處理流程：

- **Resize**: 調整圖像大小為 128x128。
- **ToTensor**: 將圖像轉換為張量，且像素值範圍從 [0, 255] 壓縮到 [0, 1]。

```
root_dir = "/kaggle/input/aoi-data/train_images/train_images"

# Create the dataset and DataLoader
dataset = GrayscaleDataset(
    csv_file='/kaggle/input/aoi-data/train.csv',
    root_dir=root_dir,
    transform=transform
)
data_loader = DataLoader(dataset, batch_size=16, shuffle=True)
```

- 設定訓練圖像路徑及 CSV 檔案。
- 創建 GrayscaleDataset 實例。
- 使用 DataLoader 將數據批量加載，批量為 16，並打亂。

```
for images, labels in data_loader:
    print(images.shape, labels.shape)  # images: [16, 1, 128, 128], labels: [16]
    break
```

- 從 data_loader 中提取一個批次的數據。
- 打印 images 和 labels 的形狀：

images: (16, 1, 128, 128)，即 16 張灰度圖像。
labels: [16]，即 16 個對應的標籤。

```python
class VGG(nn.Module):
    def __init__(self, num_classes=6):
        super(VGG, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 64, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.classifier = nn.Sequential(
            nn.Linear(256 * 16 * 16, 1024),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(1024, 512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)  # Flatten
        x = self.classifier(x)
        return x
```

- 定義基於 VGG 的 CNN 模型。
- **features**: 包含卷積層和池化層，提取圖像特徵。
- **classifier**: 包含全連接層，將特徵映射為類別。

```python
device = "cuda" if torch.cuda.is_available() else "cpu"
model = VGG(num_classes=6).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-5)
```

- 確定運算設備。
- 初始化模型，並移動到設備上。
- 定義損失函數為交叉熵損失。
- 用 Adam 優化，學習率為 10^(-5)。

```python
epochs = 100
loss_history = []
for epoch in range(epochs):
    model.train()
    for images, labels in tqdm(data_loader, desc=f"training progress"):
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        loss_history.append(loss.item())
    print(f"Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}")
torch.save(model.state_dict(), "model_state.pth")
```

- 將圖像和標籤移動到設備上。
- 更新前向傳播、計算損失和反向傳播的權重。
- 記錄損失。

```python
device = "cuda" if torch.cuda.is_available() else "cpu"
model = VGG(num_classes=6).to(device)

load_path = '/kaggle/working/model_state.pth'
if load_path is not None:
    model.load_state_dict(torch.load(load_path, weights_only=True))
    model.eval()
```

```python
# Create a test dataset and DataLoader
class TestDataset(Dataset):
    def __init__(self, csv_file, root_dir, transform=None):
        self.data_frame = pd.read_csv(csv_file)
        self.root_dir = root_dir
        self.transform = transform

    def __len__(self):
        return len(self.data_frame)

    def __getitem__(self, idx):
        img_name = os.path.join(self.root_dir, self.data_frame.iloc[idx, 0])
        image = Image.open(img_name).convert('L')  # Convert to grayscale

        if self.transform:
            image = self.transform(image)

        return image, self.data_frame.iloc[idx, 0]  # Return image and its ID

root_dir_test = "/kaggle/input/aoi-data/test_images/test_images"
test_dataset = TestDataset(
    csv_file='/kaggle/input/aoi-data/test.csv',
    root_dir=root_dir_test,
    transform=transform
)
```

- 定義測試數據類似於訓練數據，返回圖像 ID。

```python
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)
```

- 設定測試資料加載器，不打亂。

```python
predictions = []
with torch.no_grad():
    for images, ids in tqdm(test_loader, desc=f"test evaluating"):
        images = images.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        predictions.extend(zip(ids, predicted.cpu().numpy()))
```

- 不計算梯度，減少內存佔用。
- 通過測試數據進行前向推論，記錄每張圖像的 ID 和預測類別。

```python
# Save predictions to a CSV file
output_df = pd.DataFrame(predictions, columns=['ID', 'Label'])
output_df.to_csv('./test_predictions.csv', index=False)
```

- 將預測結果保存為 CSV 文件。

4. Discuss the impact of hyper-parameter

(1) Learning Rate

- Learning Rate 過大：模型可能跳過最優解，甚至發散。
- Learning Rate 過小：收斂速度非常慢，可能陷入局部最小值，導致無法達到全局最佳性能。

(2) Batch Size

- Batch Size 較小：參數更新頻繁，模型收斂速度快，但梯度波動較大，可能影響穩定性。
- Batch Size 較大：收斂更穩定，但計算開銷大，記憶體消耗高。

(3) Optimizer

- Adam：適合大多數情況，學習率自適應，能夠在不同的參數上調整學習率。

(4) Dropout 比例

- Dropout 過高：模型可能學不到足夠的特徵，導致欠擬合。
- Dropout 過低：模型可能過度依賴某些神經元，導致過度擬合。

(5) Epoch 數量

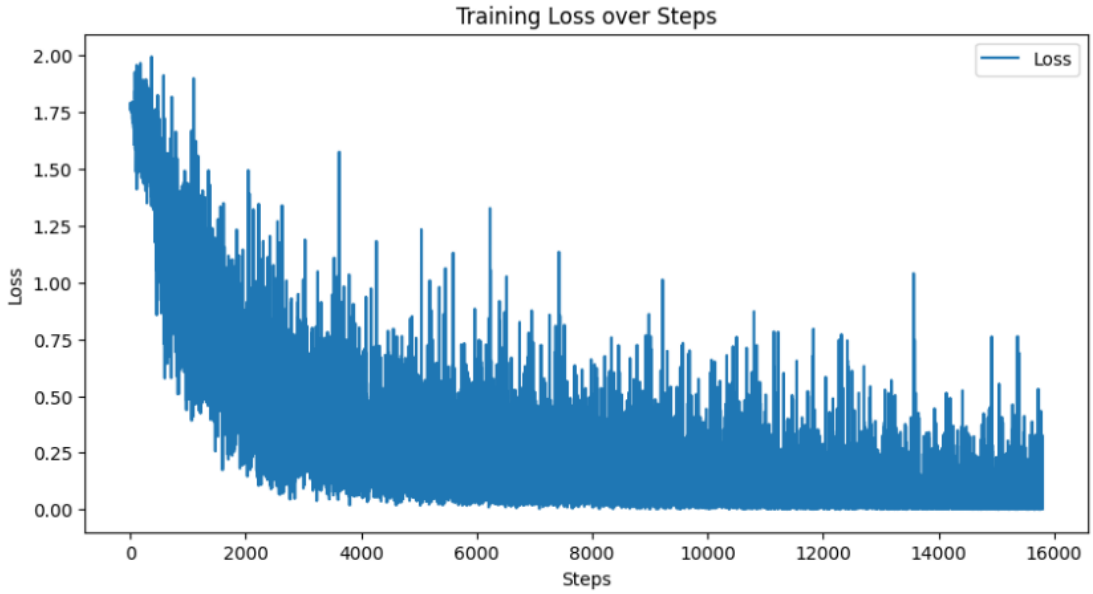
Epoch 過少：模型可能尚未學到足夠的特徵，造成欠擬合。
Epoch 過多：模型可能過度擬合，導致泛化性能下降。

5. result

<u>Score Evaluation</u>:

檔名	上傳時間	評估結果	排名
predictions.csv CHK93	2024-12-22 20:26:00	0.9699136	404/819

Accuracy = 0.9699136



- 初期：快速下降，表示模型有效學習資料的基本特徵。
- 中期：逐漸收斂，偶爾出現波動。
- 後期：穩定在較低的損失值，但仍有小範圍的波動，顯示模型在微調中尋找最佳權重。

- 整體訓練過程中沒有出現劇烈的收斂或發散，學習率適中。
- 可能的過度擬合：在後期，損失值已經非常小，表示模型可能在記憶訓練數據，而非泛化到未見過的數據上。

6. Future Direction

未來研究可以對比研究不同深度學習架構對不同的影像資料之間的優劣，優化架構來達到提高分類準確性的目的。其次對於圖像資料可以做如形狀特徵（面積、周長、圓度）、紋理特徵（使用局部二值模式、Gabor 濾波、Haralick 特徵）、位置特徵、邊緣檢測（Canny 邊緣檢測、Harris 角落檢測）等預處理。可以對比同一模型不同預處理方式對於缺陷的識別能力的影響。其次是提高算法效率使得模型在維持相當準確率的同時能夠縮短訓練模型使用的時間。為進一步增加模型的泛用性，可以通過對於訓練資料的隨機裁剪、旋轉等方式豐富訓練資料。也能夠將設定 hyper parameter 轉為根據訓練流程進行自適應性的調整。

7. Conclusion

透過這次的 Final Project，我們了解到 VGG 架構在圖像分類中的高效性，主要依賴於其深度卷積結構能夠有效地提取特徵。我們能取得高準確率，主要是因為訓練資料和測試資料分布相似，使模型能夠準確識別特徵並泛化到測試集。然而，若要進一步提升模型的泛化能力，我們需要：

1. 擴充訓練資料：增加更多多樣化的數據或使用資料增強技術。
2. 優化網絡架構：引入更高效的結構（如 ResNet）或調整現有層數。

3. 使用預訓練模型：利用大型數據集（如 ImageNet）進行預訓練再微調。
4. 正則化技術：使用 Dropout 或 L2 正則化來降低過度擬合風險。

這些策略將有助於模型在更多樣化的數據上保持穩定的表現，進一步提升分類效能。